

DAG

↳ Directed Acyclic Graph.

Top sort

① Simple

↳ Find 0 indegree

↳ delete - add edges

② DFS method:

① Check cycle not ok.

② Start DFS from any vertex.

③ Sort according to finish time.

④ Decending order.

SCC

① Kosaraju Algo

② start DFS from any vertex.

③ Graph edge reverse.

④ start DFS on reverse graph from highest finish time.

Dijkstra:

vertex $d[v]$ $P[v]$

Pseudo code

while there unvisited vertex:

mark vertex u , where d is min!

for ~~for~~ each $u \rightarrow v$

if v is not marked and $d[u] + \text{edge} < d[v]$

update v

□ Bellman-Ford:

① Use for negative value.

② run $n-1$ times.

Negative cycle don't work.

Sudo code:

do $(n-1)$ time $n = \text{no. of vertices}$:

for each $u \rightarrow v$

if $d[u] + \text{edge} < d[v]$

update $d[v]$

□ MST

① All vertices and $(n-1)$ edges.

$$S.T = n^{n-2}$$

MST 2 method

① Kruskal

② Prim.

i) Sort all edges and pick min

ii) if cycle occurs edge discard.

↳ mainly not pick that edge.

Disjoint set union.

Union set not used second time.

Denifit Don't draw the graph.

Prim:

i) Pick min edge, From particular vertices.

ii) ~~But~~ Pick min from all

2 Special Terms:

(1) Dense Graph: $\rightarrow$ Prim better.

(2) Sparse Graph: $\rightarrow$ all.

Greedy $\rightarrow$

- (i) Interval Scheduling
- (ii) Fractional knapsack
- (iii) Huffman coding.

(1) Interval Scheduling

Sort by finish time.

(i) Pick task according to finish time.

(2) Fractional knapsack

(i) (price/weight)

(ii) highest (P/W) $\rightarrow$ pick.

Huffman encoding/decoding :

- ① Count letter and sort them.
- ② merge 2 vertices (min counted)

Dynamic Programming:

- (i) Overlapping subproblems: By solving sub problem ~~main solve~~.
- (ii) Optimal substructure: each subproblems will be solved in same problem method.

① LCS

② Binary knap sack

③ Coin change.

Sub code:

① if $s[i] = s[j]$

$$LCS[i, j] = 1 + LCS[i-1, j-1]$$

else:

$$LCS[i, j] = \max(LCS[i-1, j], LCS[i, j-1])$$

7) Binary Knapsack:

↳ no fractional

sudo code:

if item weight > capacity

copy top

else:

max(top, item price + dp [prev row]

[capacity - item weight]

column]

7) Coin Change Problem:

sudo code

if coin-price > amount

copy-top

else:

min(top, 1 + DP [same row]

[amount - coin price] column,